HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO THỰC HÀNH
DỊCH NGƯỢC
AN TOÀN GAME
BENTOLOOP

Giảng viên hướng dẫn: Cao Minh Thắng
Lớp: D24CQGA02-B

Sinh viên **MSV**
Nguyễn Thành Vinh B24DCGA162

HÀ NỘI - 10/2026

MỤC LỤC

1 Mục tiêu và kết quả thực hành 2
2 Decompile APK và lấy thư viện native 2
3 Xác định hàm kiểm tra license từ dữ liệu dump 4
4 Import thư viện và gắn nhãn trong Ghidra 5
4.1 Import libil2cpp và kiểm tra Image Base 5
4.2 Gắn tên từ script json 6
5 Phân tích điều kiện chấp nhận license 7
5.1 Xác định chuỗi được đem so sánh trong VerifyLicense 7
5.2 Theo nhánh điều kiện đến giá trị trả về 8
5.3 Rút gọn luồng kiểm tra để chọn bước tiếp theo 8
6 Tìm key và kiểm chứng trên APK gốc 8
6.1 Mở StringLiteral 2770 trong Listing 8
6.2 Đối chiếu địa chỉ và giá trị trong stringliteral json 9
6.3 Nhập key và kiểm tra kết quả trên APK gốc 10
7 Sửa VerifyLicense và xuất thư viện 11
7.1 Chuẩn bị bản sao và chọn lệnh cần sửa 11
7.2 Thay hai lệnh để trả về thành công ngay 11
7.3 Đọc lại Decompiler sau khi patch 11
7.4 Xuất file ELF chứa byte đã sửa 12
7.5 Thay thư viện trong thư mục APK đã Decompile 13
8 Build APK và kiểm thử bản đã sửa 13
8.1 Cập nhật công cụ build 13
8.2 Làm mới framework và Compile 14
8.3 Lấy APK đã build và ký 15
8.4 Kiểm thử với chuỗi khác key đã tìm được 16
9 Nhận xét và phương án phòng tránh 16
9.1 Nguyên nhân cơ chế có thể bị vòng tránh 16
9.2 Các biện pháp giảm rủi ro 17
9.3 Kết luận thực hành 17
Tài liệu tham khảo 17

1 Mục tiêu và kết quả thực hành

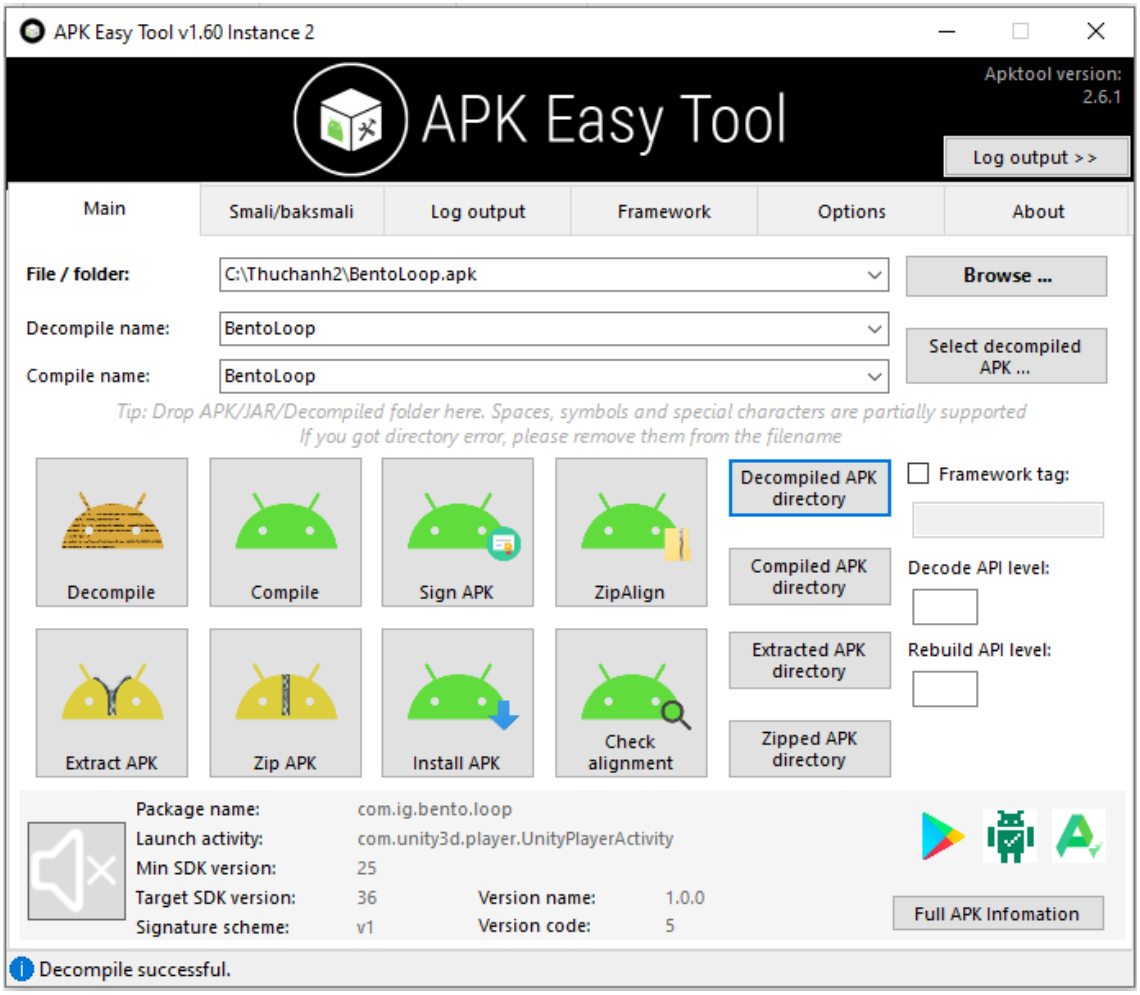
Bài thực hành phân tích cơ chế kiểm tra license của BentoLoop, tìm key được chấp nhận và sửa thư viện native để build lại APK. Kết quả tìm được là BENTO-EASY-2025. APK gốc báo kích hoạt thành công khi nhập key này và chuyển vào Level 1. Sau khi sửa VerifyLicense để trả về thành công ngay đầu hàm, APK được build và ký lại bằng APK Easy Tool. Bản APK mới báo kích hoạt thành công với chuỗi vinh123.

Quá trình thực hiện gồm giải mã cấu trúc APK, tra cứu dữ liệu IL2CPP, phân tích mã native trong Ghidra, sửa hai lệnh ARM64 và kiểm thử ứng dụng. Các ảnh minh chứng được đặt cùng phần giải thích tương ứng để theo dõi dữ liệu đầu vào, thao tác và kết quả của từng bước.

Thành phần	Thông tin sử dụng
Môi trường	Windows trong máy ảo VMware Workstation; kiểm thử trên thiết bị Android kết nối với máy thật
Ứng dụng	BentoLoop.apk; package com.ig.bento.loop; version name 1.0.0; version code 5
Thư viện phân tích	lib/arm64-v8a/libil2cpp.so; kiến trúc AARCH64, little-endian, 64 bit
APK Easy Tool	Phiên bản 1.60; Decompile bằng Apktool 2.6.1; Compile bằng Apktool 2.12.1
Ghidra	Phiên bản 12.1.2; project Thuchanh2; Image Base 0x100000
Dữ liệu dump	Gói libil2cpp_il2cppdump_20261007_0259 được cung cấp trong bài thực hành

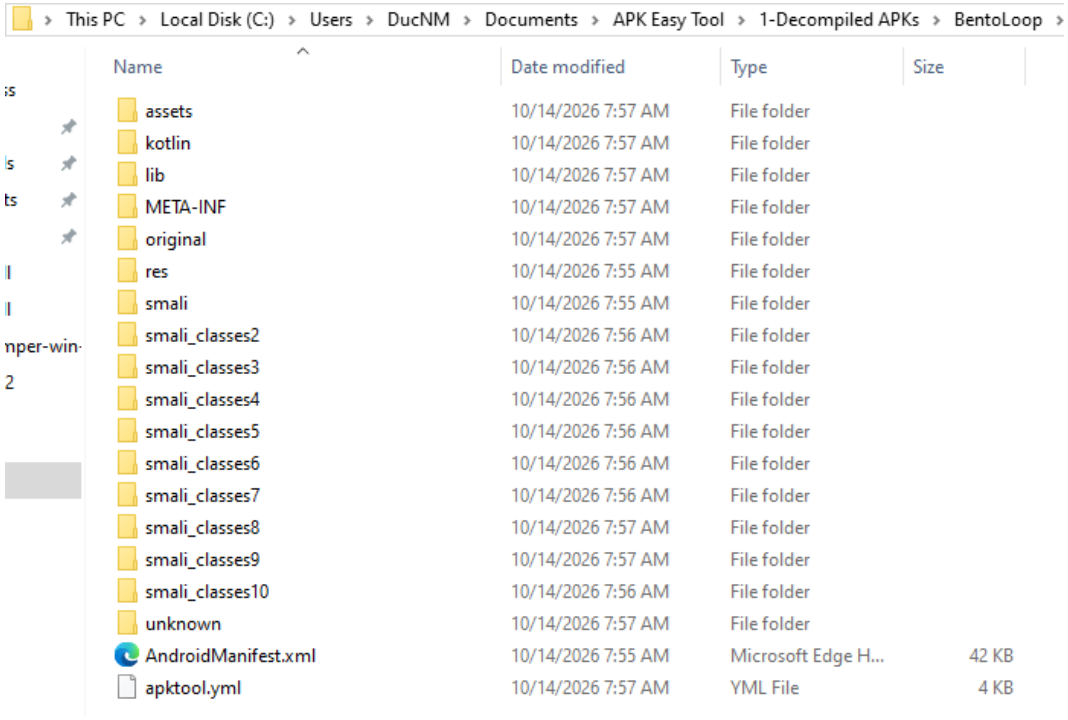
2 Decompile APK và lấy thư viện native

File đầu vào là C:\Thuchanh2\BentoLoop.apk trong máy ảo. Tại APK Easy Tool, Browse được dùng để chọn file, sau đó bấm Decompile. Công cụ hiển thị Decompile successful và nhận dạng package com.ig.bento.loop. APK khai báo min SDK 25 và target SDK 36.



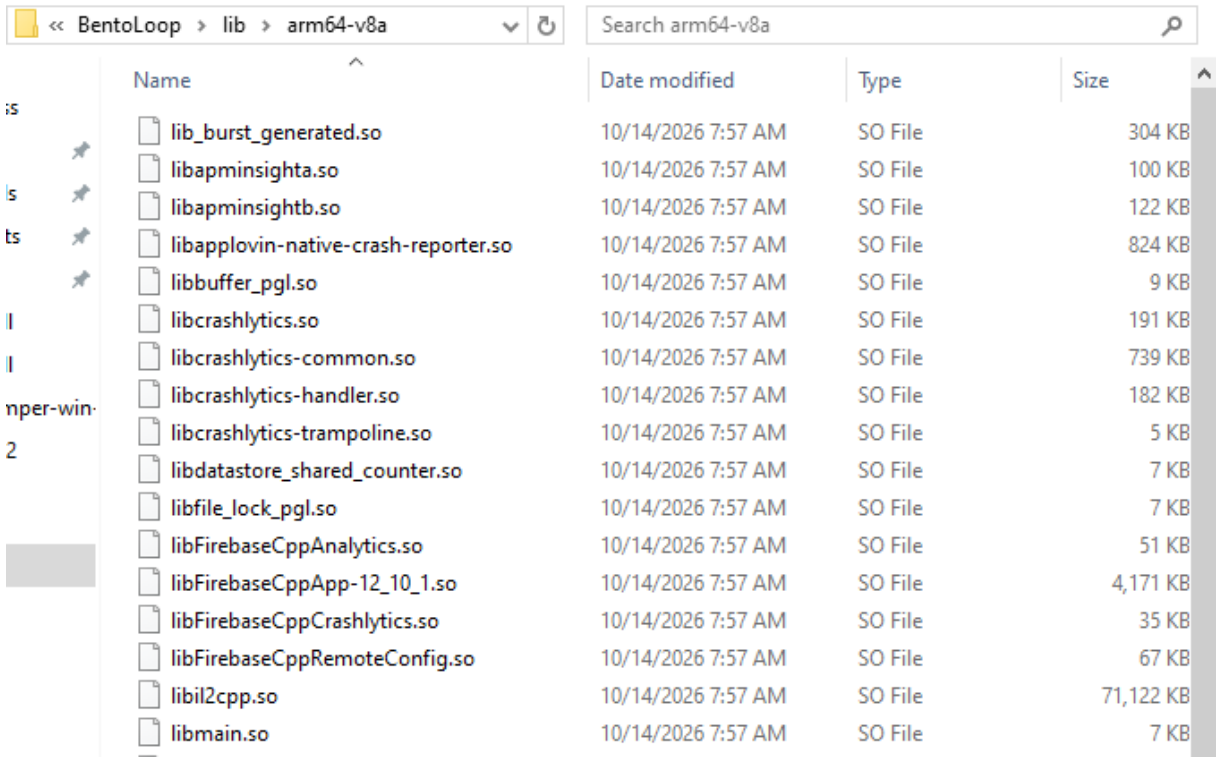
Hình 1. Decompile BentoLoop.apk bằng APK Easy Tool

Kết quả nằm trong thư mục BentoLoop dưới 1-Decompiled APKs. Thư mục có assets, lib, res, các thư mục smali, AndroidManifest.xml và apktool.yml. File apktool.yml giúp công cụ nhận diện thư mục này khi Compile lại.



Hình 2. Cấu trúc thư mục BentoLoop sau Decompile

Thư viện được chọn để phân tích là libil2cpp.so trong lib\arm64-v8a. Đây là mã native của phần IL2CPP trong ứng dụng. File có dung lượng hiển thị 71.122 KB, tương ứng 72.828.928 byte. Ở bước sửa hàm, chương trình được lưu thành bản sao trong Ghidra để giữ phần phân tích của bản gốc.



Hình 3. Thư viện libil2cpp.so của bản ARM64

```
Thư mục làm việc
C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop

Thư viện cần thay khi build
lib\arm64-v8a\libil2cpp.so
```

3 Xác định hàm kiểm tra license từ dữ liệu dump

Bộ dump được cung cấp gồm dump.cs, script.json, stringliteral.json, các script hỗ trợ Ghidra và thư mục DummyDll. Trong bước này, dữ liệu dump được dùng để tra cứu tên lớp và địa chỉ; thư viện native được lấy từ APK đã Decompile.

Tìm LicenseVerifier trong dump.cs cho kết quả lớp static thuộc namespace GeckoArrow.Runtime.LoadingSystem. VerifyLicense nhận string inputKey và trả về bool. Lớp còn có CheckTransformedLicense, IsLicenseActivated, SaveLicenseActivated và field EncryptedBytes. Thân hàm trong dump.cs không chứa logic thực thi nên cần mở thư viện trong Ghidra để đọc mã native.

```
// Namespace: GeckoArrow.Runtime.LoadingSystem
public static class LicenseVerifier // TypeDefIndex: 12855
{
        // Fields
        private static readonly byte[] EncryptedBytes; // 0x0

        // Methods

        // RVA: 0x1E76FA0 Offset: 0x1E72FA0 VA: 0x1E76FA0
        public static bool CheckTransformedLicense(string key) { }

        // RVA: 0x1E7689C Offset: 0x1E7289C VA: 0x1E7689C
        public static bool VerifyLicense(string inputKey) { }

        // RVA: 0x1E76EF4 Offset: 0x1E72EF4 VA: 0x1E76EF4
        public static bool IsLicenseActivated() { }

        // RVA: 0x1E769FC Offset: 0x1E729FC VA: 0x1E769FC
        public static void SaveLicenseActivated(bool activated = True) { }

        // RVA: 0x1E77110 Offset: 0x1E73110 VA: 0x1E77110
        private static void .cctor() { }
}
```

Hình 4. Lớp LicenseVerifier và địa chỉ các phương thức trong dump.cs

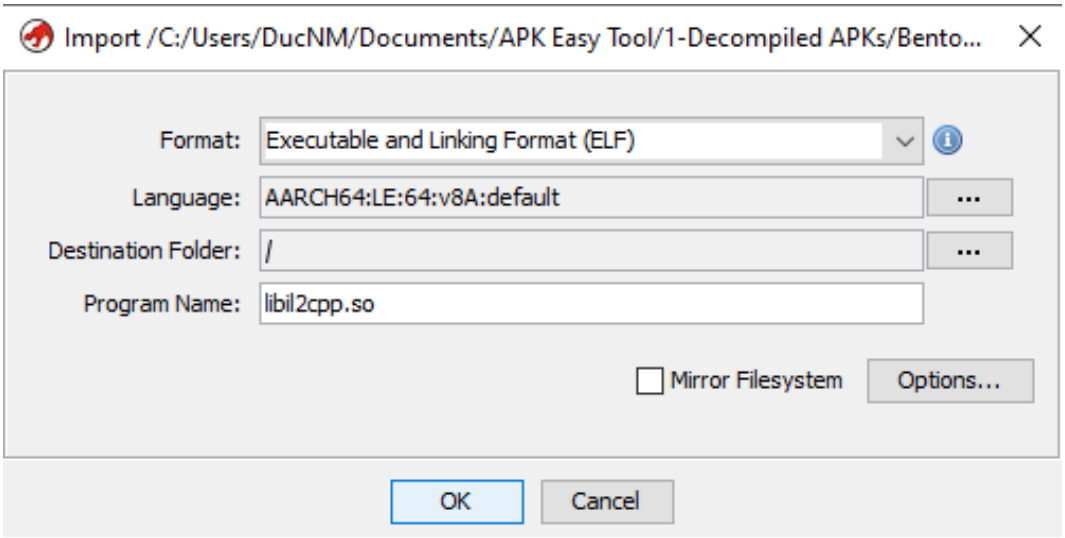
Phương thức	RVA	Địa chỉ trong Ghidra
VerifyLicense	0x1E7689C	0x1F7689C
CheckTransformedLicense	0x1E76FA0	0x1F76FA0
IsLicenseActivated	0x1E76EF4	0x1F76EF4
SaveLicenseActivated	0x1E769FC	0x1F769FC
.cctor	0x1E77110	0x1F77110

RVA là địa chỉ tương đối so với Image Base. Cột địa chỉ Ghidra trong bảng đã cộng Image Base 0x100000. Với VerifyLicense, dump còn ghi File Offset 0x1E7289C. Offset này chỉ vị trí trong file và khác địa chỉ hiển thị trong Ghidra.

4 Import thư viện và gắn nhãn trong Ghidra

4.1 Import libil2cpp và kiểm tra Image Base

libil2cpp.so được import vào project Thuchanh2. Ghidra nhận dạng file ELF với language AARCH64:LE:64:v8A:default. Sau khi mở CodeBrowser, phân tích tự động được chạy để nhận diện các hàm và tham chiếu.



Hình 5. Ghidra nhận dạng thư viện ELF ARM64

Memory Map hiển thị Image Base 00100000. Vì vậy, địa chỉ cần nhập trong Go To được tính bằng RVA cộng 0x100000. Phép quy đổi này được dùng nhất quán khi tìm hàm và đối chiếu chuỗi.

```
Địa chỉ Ghidra = Image Base + RVA
0x00100000 + 0x01E7689C = 0x01F7689C
```

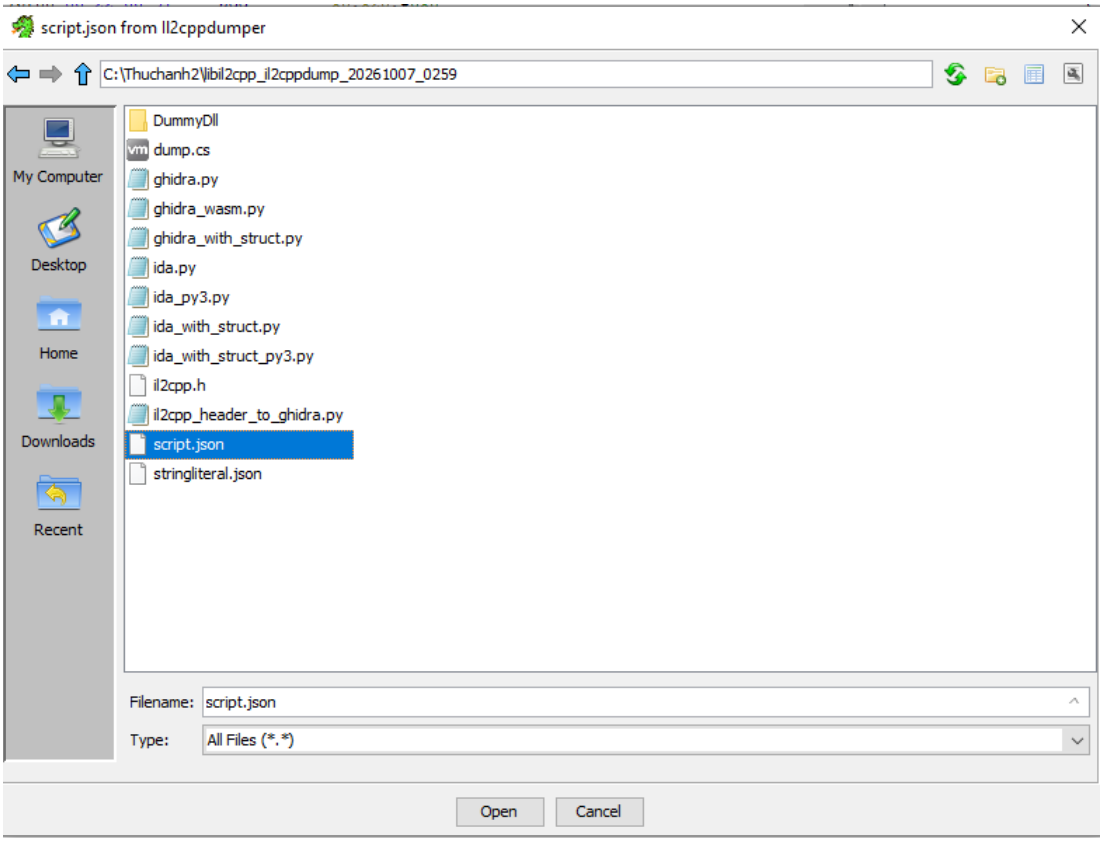
Memory Map - Image Base: 00100000

Name	Start	End	Length	R	W	X	Volatile	Artificial	Overlayed
segment_1.1	00100000	0010026f	0x270	☑	☐	☐	☐	☐	
.note.android.ident	00100270	00100307	0x98	☑	☐	☐	☐	☐	
.note.gnu.build-id	00100308	0010032b	0x24	☑	☐	☐	☐	☐	
.dynsym	00100330	0011001f	0xfcf0	☑	☐	☐	☐	☐	
.gnu.version	00110020	00111533	0x1514	☑	☐	☐	☐	☐	
.gnu.version_r	00111534	00111593	0x60	☑	☐	☐	☐	☐	
.gnu.hash	00111598	001154d7	0x3f40	☑	☐	☐	☐	☐	
.dynstr	001154d8	00131422	0x1bf4b	☑	☐	☐	☐	☐	
.rela.dyn	00131428	00b9910f	0xa67ce8	☑	☐	☐	☐	☐	
.rela.plt	00b99110	00b9c877	0x3768	☑	☐	☐	☐	☐	
.gcc_except_table	00b9c878	00c47fab	0xab734	☑	☐	☐	☐	☐	
.rodata	00c47fb0	010d2a4b	0x48aa9c	☑	☐	☐	☐	☐	
.eh_frame_hdr	010d2a4c	012c07b7	0x1edd6c	☑	☐	☐	☐	☐	
.eh_frame	012c07b8	01b22b8f	0x8623d8	☑	☐	☐	☐	☐	
.text	01b26b90	01e96f6b	0x3703dc	☑	☐	☑	☐	☐	
il2cpp	01e96f6c	04181d67	0x22eadfc	☑	☐	☑	☐	☐	
.plt	04181d70	0418427f	0x2510	☑	☐	☑	☐	☐	

Hình 6. Memory Map xác nhận Image Base 0x100000

4.2 Gắn tên từ script json

Thư mục chứa ghidra.py được thêm vào Script Manager. Script được chạy với file script.json trong gói dump BentoLoop. Kết quả gắn tên giúp Decompiler hiển thị các phương thức như LicenseVerifier$$VerifyLicense, String.Trim và String.Equals thay cho phần lớn tên tự động.



Hình 7. Chọn script.json của BentoLoop để gắn tên trong Ghidra

FUN_01f7689c là tên tự động của Ghidra tại địa chỉ 0x1F7689C. Sau khi áp dụng dữ liệu dump, cùng vị trí đó có tên GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$VerifyLicense. Nhãn được đổi để dễ đọc mã giả; mã máy chỉ thay đổi ở bước patch sau đó.

5 Phân tích điều kiện chấp nhận license

5.1 Xác định chuỗi được đem so sánh trong VerifyLicense

Mở VerifyLicense tại 0x1F7689C bằng Go To và đọc phần xử lý đầu vào trong Decompiler. Đoạn cần theo dõi là các lời gọi IsNullOrWhiteSpace, Trim và Equals trong ảnh dưới.

```
18  }
19  uVar2 = System.String$$IsNullOrWhiteSpace(param_1,0);
20  if ((uVar2 & 1) == 0) {
21    if (param_1 == 0) {
22                    /* WARNING: Subroutine does not return */
23      FUN_01da5340();
24    }
25    uVar3 = System.String$$Trim(param_1,0);
26    uVar2 = System.String$$Equals(uVar3,StringLiteral_2770,4,0);
27    if ((uVar2 & 1) == 0) {
```

Hình 8. Các dòng xử lý đầu vào và lời gọi String.Equals với StringLiteral_2770

Trong Hình 8, param_1 là chuỗi được truyền vào hàm. Dòng 19 gọi System.String$$IsNullOrWhiteSpace để kiểm tra chuỗi rỗng hoặc chỉ có khoảng trắng. Điều kiện ở dòng 20 chỉ đưa luồng vào phần kiểm tra tiếp khi kết quả bằng 0. Dòng 25 gọi System.String$$Trim và lưu chuỗi đã bỏ khoảng trắng ở hai đầu vào uVar3.

Dòng 26 là vị trí cần chú ý: System.String$$Equals(uVar3, StringLiteral_2770, 4, 0). Chuỗi vừa xử lý được so sánh với StringLiteral_2770, còn kết quả so sánh được lưu vào uVar2. StringLiteral_2770 là nhãn dữ liệu chuỗi trong Ghidra, không phải một hàm. Ở bước này, đầu mối thu được là nhãn của chuỗi cần đọc để tìm key.

5.2 Theo nhánh điều kiện đến giá trị trả về

Sau phép Equals, đọc tiếp các nhánh if và phần cuối VerifyLicense để xác định chuỗi khớp có thực sự dẫn đến kết quả thành công hay không.

```
39        iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
40        puVar4 = &StringLiteral_11021;
41      }
42    }
43    else {
44      iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
45      puVar4 = &StringLiteral_11020;
46    }
47    if (iVar1 == 0) {
48      thunk_FUN_01e01e10();
49    }
50    UnityEngine.Debug$$Log(*puVar4,0);
51    uVar3 = 1;
52  }
53  else {
54LAB_01f76914:
55    uVar3 = 0;
56  }
57  return uVar3;
58}
59
```

Hình 9. Phần cuối VerifyLicense gán kết quả thành công bằng 1 và thất bại bằng 0

Khi phép so sánh trực tiếp không khớp, VerifyLicense gọi CheckTransformedLicense và có thể gọi tiếp SecurityAuthService.DispatchVerification. Nếu các nhánh bổ sung đều không chấp nhận đầu vào, luồng chuyển tới LAB_01f76914. Trong Hình 9, nhãn này gán uVar3 = 0 rồi đi đến return uVar3, tương ứng kết quả false.

Nhánh so sánh trực tiếp khớp đi đến phần ghi log và phép gán uVar3 = 1 ở dòng 51. Dòng 57 trả về giá trị này, tương ứng true. Vì vậy, việc lần đến StringLiteral_2770 dựa trên cả lời gọi so sánh ở Hình 8 và đường trả về thành công ở Hình 9, chứ không chỉ dựa vào tên của nhãn.

5.3 Rút gọn luồng kiểm tra để chọn bước tiếp theo

Các điều kiện quan sát được có thể viết lại như sau. Phần khởi tạo dữ liệu và ghi log được lược đi để thấy rõ thứ tự các nhánh chấp nhận đầu vào.

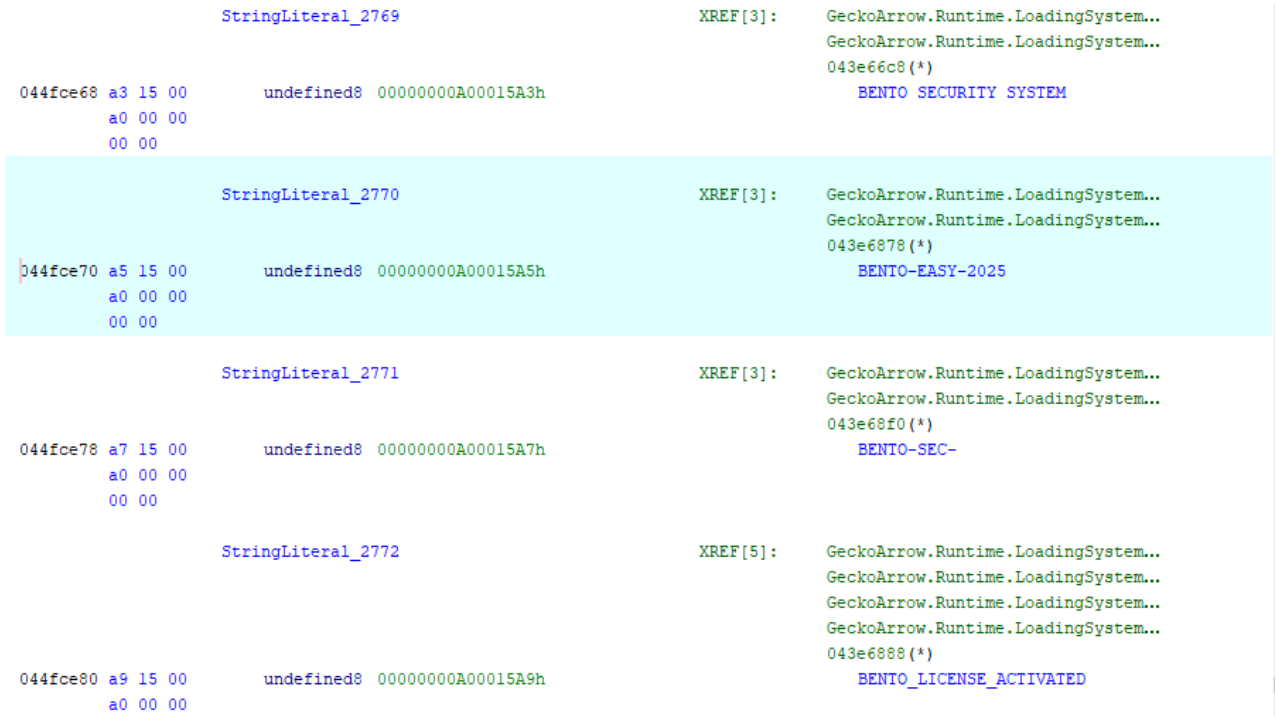
```
if (IsNullOrWhiteSpace(inputKey)) return false;
key = Trim(inputKey);
if (Equals(key, StringLiteral_2770, 4)) return true;
if (CheckTransformedLicense(key)) return true;
return SecurityAuthService.DispatchVerification(key);
```

Luồng trên cho thấy cần đọc nội dung StringLiteral_2770 trước. Nhánh CheckTransformedLicense còn kiểm tra từng ký tự với mảng byte bằng XOR 0x53 và ((i × 13) AND 0xFF). Tuy nhiên, nhánh Equals đã chỉ ra một chuỗi có thể được chấp nhận trực tiếp, nên bước tiếp theo tập trung vào nhãn 2770 và kiểm chứng chuỗi đó trên APK gốc.

6 Tìm key và kiểm chứng trên APK gốc

6.1 Mở StringLiteral 2770 trong Listing

Từ lời gọi Equals ở Hình 8, bấm đúp StringLiteral_2770 trong Decompiler. Listing chuyển đến vùng dữ liệu có nhãn này. Ảnh dưới giữ cả địa chỉ, nhãn và phần chú thích để đối chiếu cùng một vị trí.



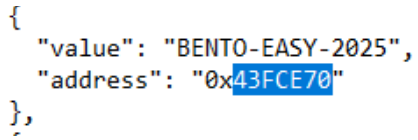
Hình 10. StringLiteral_2770 tại 044fce70 và nội dung BENTO-EASY-2025 trong Listing

Trong Hình 10, vùng được chọn màu xanh nhạt mang nhãn StringLiteral_2770. Cột địa chỉ bên trái hiển thị 044fce70, còn chú thích bên phải hiển thị BENTO-EASY-2025. Đây chính là nhãn được truyền vào Equals ở Hình 8. Kết quả thu được ở bước này là chuỗi ứng viên BENTO-EASY-2025 tại địa chỉ Ghidra 0x044FCE70.

Các chuỗi lân cận giúp phân biệt vai trò của dữ liệu. StringLiteral_2769 là BENTO SECURITY SYSTEM, StringLiteral_2771 là BENTO-SEC-, còn StringLiteral_2772 là BENTO_LICENSE_ACTIVATED. Chuỗi cuối dùng làm khóa lưu trạng thái kích hoạt. Chuỗi cần kiểm chứng là BENTO-EASY-2025 vì được tham chiếu trực tiếp trong phép so sánh của VerifyLicense.

6.2 Đối chiếu địa chỉ và giá trị trong stringliteral json

Mở stringliteral.json thuộc cùng gói dump và tìm địa chỉ 43FCE70. Mục được tìm thấy phải chứa cả value và address, như ảnh dưới.



Hình 11. Mục JSON có value BENTO-EASY-2025 và address 0x43FCE70

Hình 11 cho thấy value là BENTO-EASY-2025, trùng chú thích tại StringLiteral_2770 trong Ghidra. address là 0x43FCE70, thấp hơn địa chỉ Listing một lượng 0x100000. Đây là Image Base đã kiểm tra ở Memory Map, nên hai địa chỉ được quy đổi như sau.

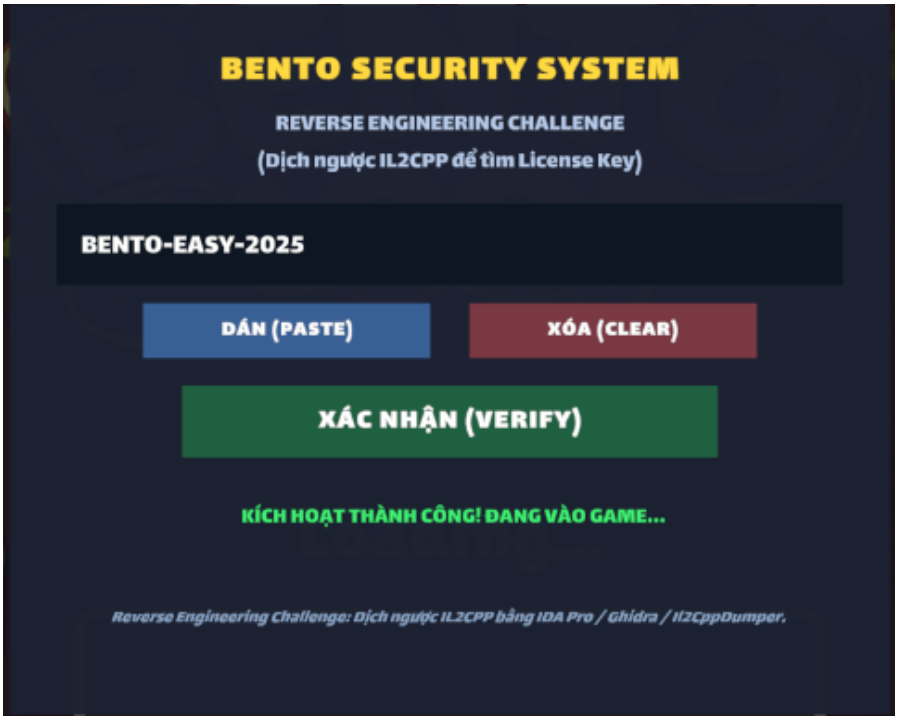
```
Địa chỉ trong JSON       0x043FCE70
Image Base của Ghidra    0x00100000
Địa chỉ trong Listing   0x044FCE70

0x043FCE70 + 0x00100000 = 0x044FCE70
```

Ghidra hiển thị 044fce70, còn JSON ghi 0x43FCE70; khác biệt này được giải thích bằng Image Base. Giá trị chuỗi và địa chỉ sau quy đổi đều khớp. Đối chiếu JSON xác nhận nguồn dữ liệu của nhãn, còn việc chuỗi có được chấp nhận hay không được kiểm tra bằng chạy ứng dụng ở bước sau.

6.3 Nhập key và kiểm tra kết quả trên APK gốc

Mở APK gốc trên thiết bị Android, nhập BENTO-EASY-2025 rồi bấm XÁC NHẬN (VERIFY). Ảnh dưới ghi lại key đã nhập và thông báo sau khi xác nhận.



Hình 12. APK gốc báo kích hoạt thành công sau khi nhập BENTO-EASY-2025

Trong Hình 12, ô nhập giữ nguyên BENTO-EASY-2025 và phía dưới hiện thông báo màu xanh KÍCH HOẠT THÀNH CÔNG! ĐANG VÀO GAME... Kết quả này xác nhận chuỗi tìm được trong Listing và JSON được APK gốc chấp nhận.



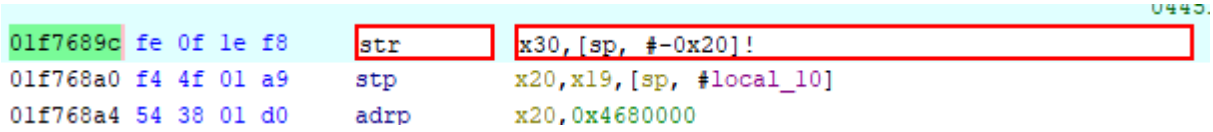
Hình 13. APK gốc chuyển vào màn chơi Level 1 sau khi xác nhận key

Hình 13 cho thấy màn chơi Level 1 xuất hiện sau màn hình license. Hai ảnh ghi nhận hai trạng thái liên tiếp: ứng dụng báo chấp nhận key và trò chơi đã vào màn chơi. Đây là kết quả hoàn thành phần tìm key của bài thực hành.

7 Sửa VerifyLicense và xuất thư viện

7.1 Chuẩn bị bản sao và chọn lệnh cần sửa

Trong Ghidra, lưu chương trình thành bản sao libil2cpp_patched để giữ phần phân tích của bản gốc. Mở lại VerifyLicense tại 0x1F7689C. Chọn lệnh đầu trong Listing rồi mở Patch Instruction; vùng nhập lệnh được viền đỏ như trong ảnh.

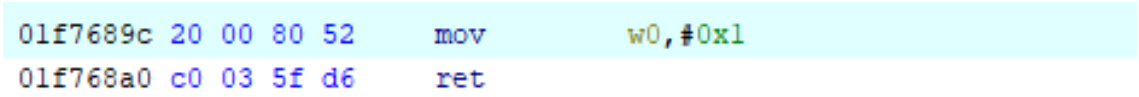


Hình 14. Vùng Patch Instruction tại lệnh đầu VerifyLicense trước khi thay mã

Hình 14 hiển thị lệnh str x30, [sp, #-0x20]! tại 01f7689c. Ngay bên dưới, tại 01f768a0, là lệnh stp x20, x19, [sp, #0x10]. Đây là hai lệnh đầu của hàm trước khi patch. Địa chỉ cần chọn là 0x1F7689C trong chương trình Ghidra; File Offset 0x1E7289C trong dump là vị trí trong file, không dùng thay cho địa chỉ Go To ở bước này.

7.2 Thay hai lệnh để trả về thành công ngay

Tại 0x1F7689C, thay lệnh str bằng mov w0, #0x1. Tại 0x1F768A0, thay lệnh stp bằng ret. Sau khi xác nhận hai thao tác Patch Instruction, Listing hiển thị mã mới như ảnh dưới.



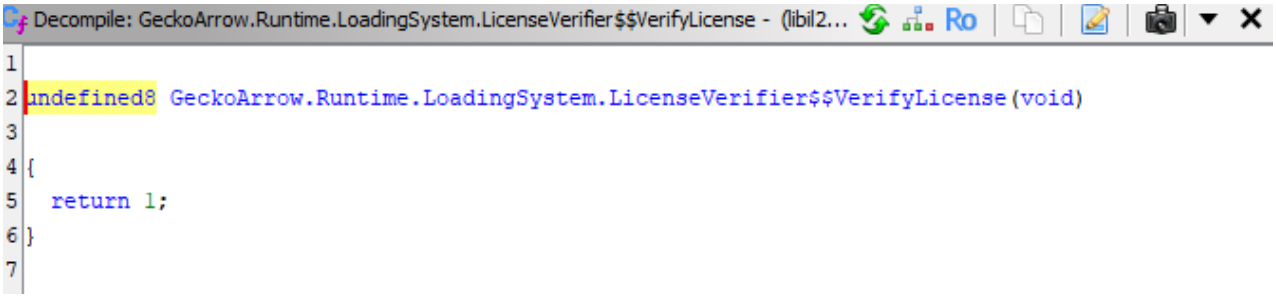
Hình 15. Hai lệnh ARM64 sau patch là mov w0, #0x1 và ret

Trong Hình 15, địa chỉ 01f7689c có các byte 20 00 80 52 và lệnh mov w0, #0x1. Địa chỉ 01f768a0 có các byte C0 03 5F D6 và lệnh ret. Lệnh mov đặt giá trị trả về bằng 1; ret kết thúc hàm ngay sau đó. Hai lệnh đều dài 4 byte, nên phần sửa chiếm 8 byte đầu của VerifyLicense.

Địa chỉ Ghidra	Lệnh trước khi sửa	Lệnh sau khi sửa
0x1F7689C	str x30, [sp, #-0x20]!	mov w0, #0x1
0x1F768A0	stp x20, x19, [sp, #0x10]	ret

7.3 Đọc lại Decompiler sau khi patch

Sau khi sửa Listing, xem lại cùng hàm trong Decompiler để kiểm tra tác động của mã mới.



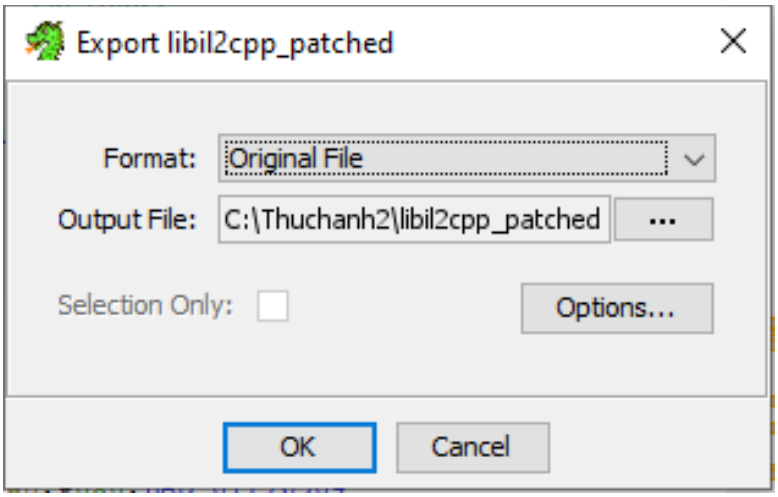
Hình 16. Decompiler của VerifyLicense sau patch chỉ còn return 1

Hình 16 hiển thị tên LicenseVerifier$$VerifyLicense và thân hàm return 1. So với Hình 8 và Hình 9, luồng hiện tại không còn đi qua IsNullOrWhiteSpace, Trim, Equals hoặc các nhánh kiểm tra bổ sung. Ghidra có thể đổi phần tham số hiển thị thành void vì mã mới không sử dụng đầu vào; tên nhãn vẫn xác định đúng hàm đã sửa.

Mã trả về được đặt trước các lệnh cấp phát stack và gọi hàm con, nên hàm kết thúc mà chưa làm thay đổi stack theo phần thân cũ. Các lệnh còn lại vẫn có thể xuất hiện phía dưới trong Listing nhưng không được thực thi từ điểm vào này. Kết quả ở Ghidra là bằng chứng phần mã đã thay đổi; hiệu lực trên ứng dụng được kiểm tra sau khi xuất thư viện và build APK ở phần 8.

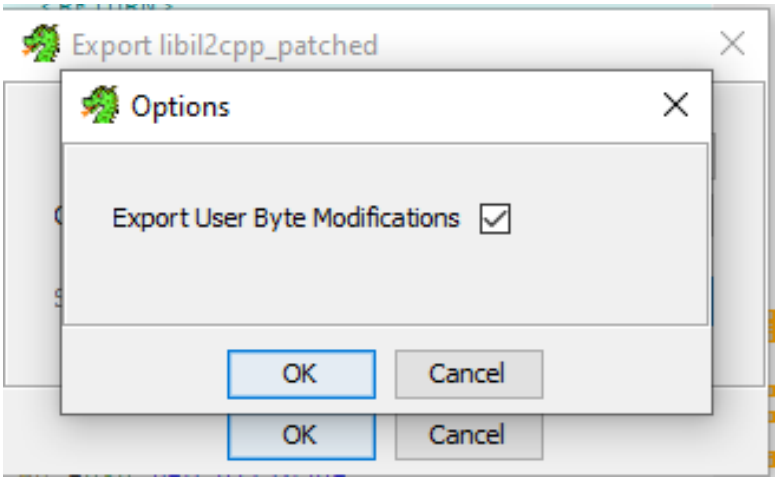
7.4 Xuất file ELF chứa byte đã sửa

Lưu chương trình rồi mở File → Export Program. Chọn Format là Original File và đặt file đích trong C:\Thuchanh2. Ảnh dưới ghi lại định dạng xuất trước khi xác nhận.



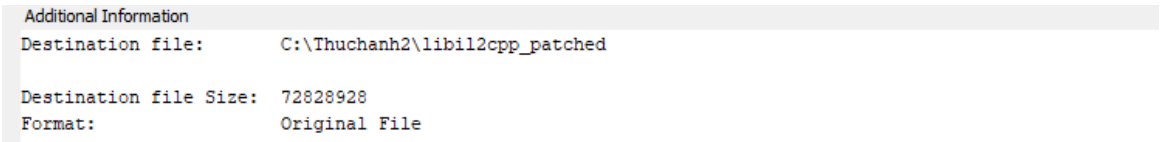
Hình 17. Chọn Original File và đường dẫn xuất thư viện đã sửa

Hình 17 hiển thị Format Original File và Output File C:\Thuchanh2\libil2cpp_patched. Việc lưu chương trình trong project chỉ giữ cơ sở dữ liệu phân tích của Ghidra; bước Export tạo ra file thư viện để thay trong thư mục APK.



Hình 18. Tích Export User Byte Modifications trong Options khi xuất

Trong Hình 18, Export User Byte Modifications đã được tích. Tùy chọn này đưa các byte vừa patch vào file xuất. Sau khi xác nhận Options và Export, kiểm tra kết quả ở Export Results Summary.

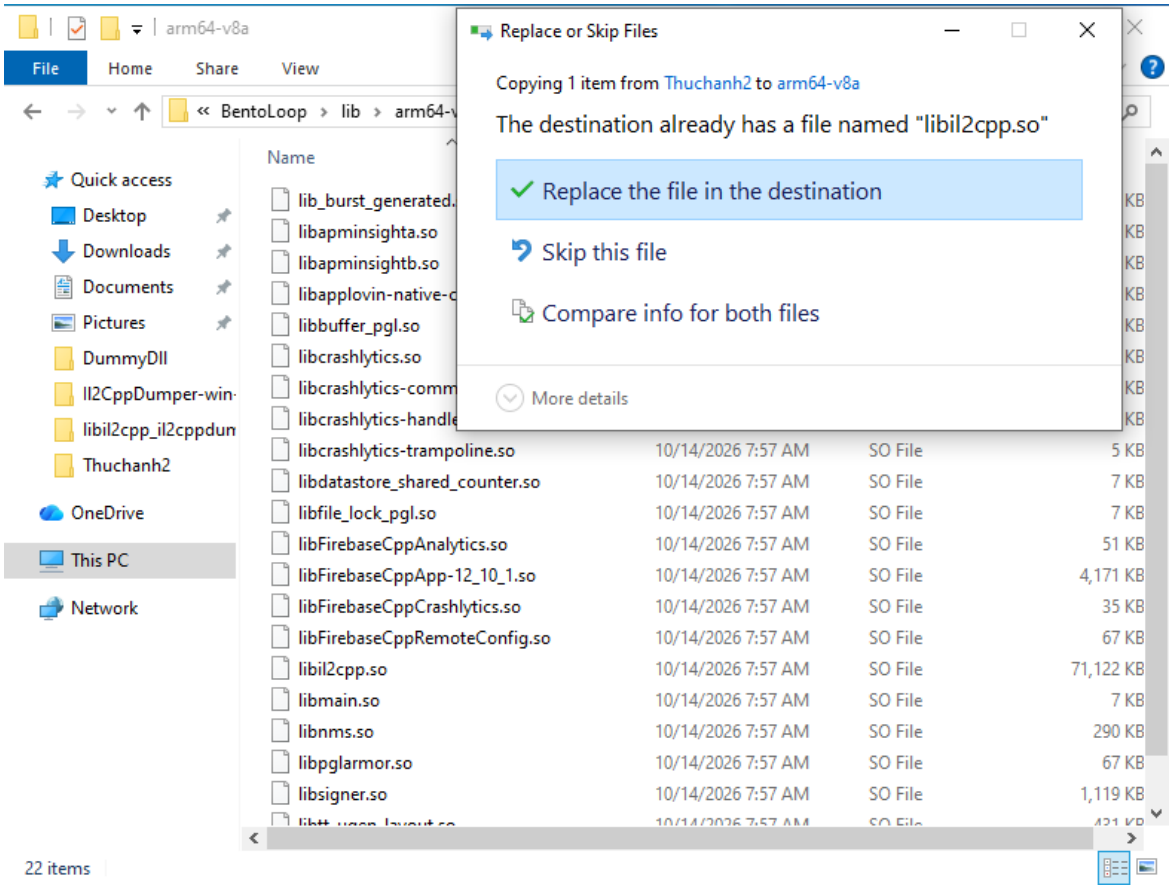


Hình 19. Kết quả xuất file đích với định dạng Original File

Hình 19 cho biết file đích là C:\Thuchanh2\libil2cpp_patched, Format là Original File và kích thước 72.828.928 byte. File xuất được đổi tên thành libil2cpp.so để khớp tên thư viện trong APK. Kích thước được giữ nguyên phù hợp với việc thay 8 byte tại chỗ, không chèn thêm lệnh vào file.

7.5 Thay thư viện trong thư mục APK đã Decompile

Copy libil2cpp.so vừa xuất vào lib\arm64-v8a của thư mục BentoLoop đã Decompile. Windows báo đã có file cùng tên, như trong ảnh dưới.



Hình 20. Xác nhận thay libil2cpp.so tại thư mục arm64-v8a

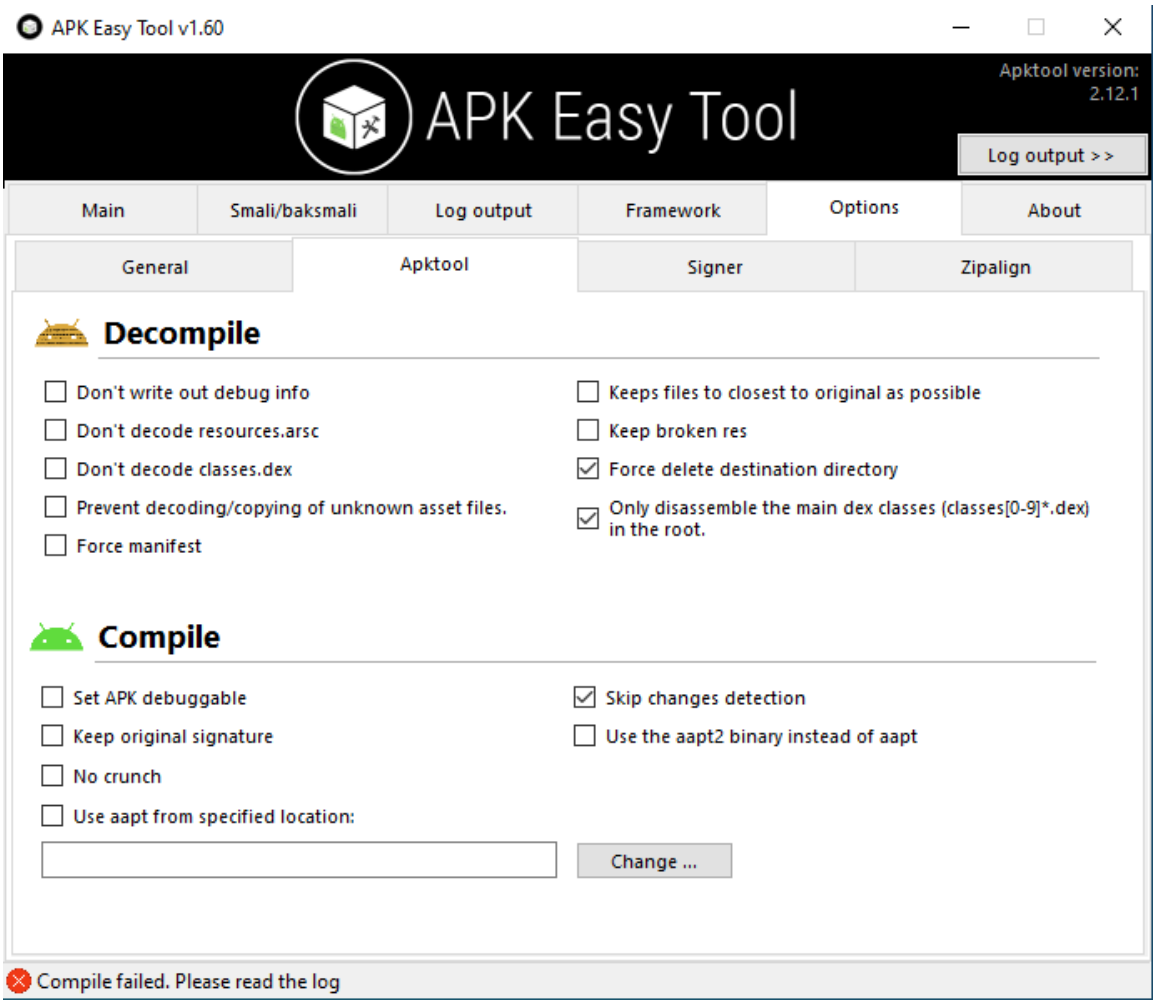
Trong Hình 20, hộp thoại ghi rõ file trùng tên là libil2cpp.so và thư mục đích là arm64-v8a. Chọn Replace the file in the destination để thay thư viện cũ bằng bản đã patch. Sau khi copy hoàn tất, thư mục BentoLoop chứa thư viện mới được dùng làm đầu vào Compile ở phần 8. Phạm vi thay thư viện trong bài là bản ARM64.

8 Build APK và kiểm thử bản đã sửa

8.1 Cập nhật công cụ build

Trong APK Easy Tool, Select decompiled APK được dùng để chọn thư mục BentoLoop chứa apktool.yml, sau đó bấm Compile. Lần build đầu với Apktool 2.6.1 dừng ở manifest do không nhận thẻ property. Thư viện đã patch không phải thành phần gây ra lỗi này.

Apktool 2.12.1 được thêm vào thư mục Apktool của APK Easy Tool và chọn trong Options → General. Giao diện cũ vẫn có thể thêm tham số --use-aapt2, khiến bản mới báo Unrecognized option. Ô Use the aapt2 binary instead of aapt trong Options → Apktool được bỏ tích. Apktool 2.12.1 sử dụng AAPT2 mặc định khi build.



Hình 21. Cấu hình Apktool 2.12.1 và bỏ tùy chọn thêm use aapt2

8.2 Làm mới framework và Compile

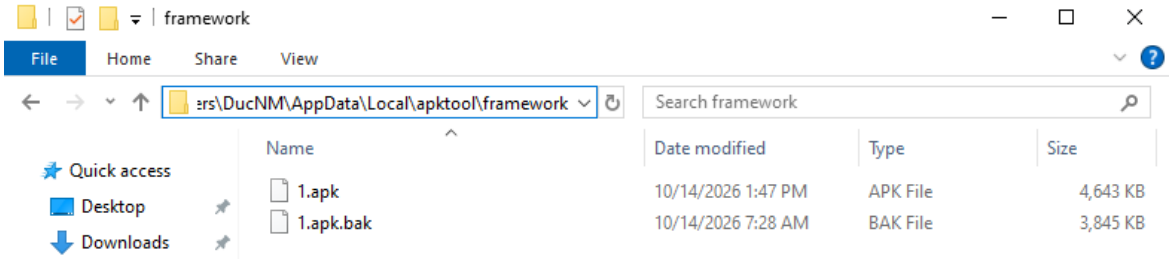
Sau khi bỏ tham số cũ, quá trình build tiếp tục nhưng báo attribute android:enableOnBackInvokedCallback not found. Log cho thấy công cụ đang dùng framework được lưu tại C:\Users\DucNM\AppData\Local\apktool\framework\1.apk. Bộ framework cũ chưa có thuộc tính này.

```
Compiling APK file... (Press ESC to cancel)
Command: java.exe -jar "C:\Thuchanh2\apk-easy-tool-1-60\Apktool\apktool_2.12.1.jar" b -f -o
"C:\Users\DucNM\Documents\APK Easy Tool\2-Recompiled APKs\BentoLoop.apk" "C:\Users\DucNM
\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop"

I: Using Apktool 2.12.1 on BentoLoop.apk with 4 threads
I: Smaling smali folder into classes.dex...
I: Smaling smali_classes2 folder into classes2.dex...
I: Smaling smali_classes10 folder into classes10.dex...
I: Building resources with aapt2...
I: Smaling smali_classes3 folder into classes3.dex...
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:85:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:126:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:127:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:128:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:129:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:130:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:165:
```

Hình 22. Lỗi thiếu enableOnBackInvokedCallback khi dùng framework cũ

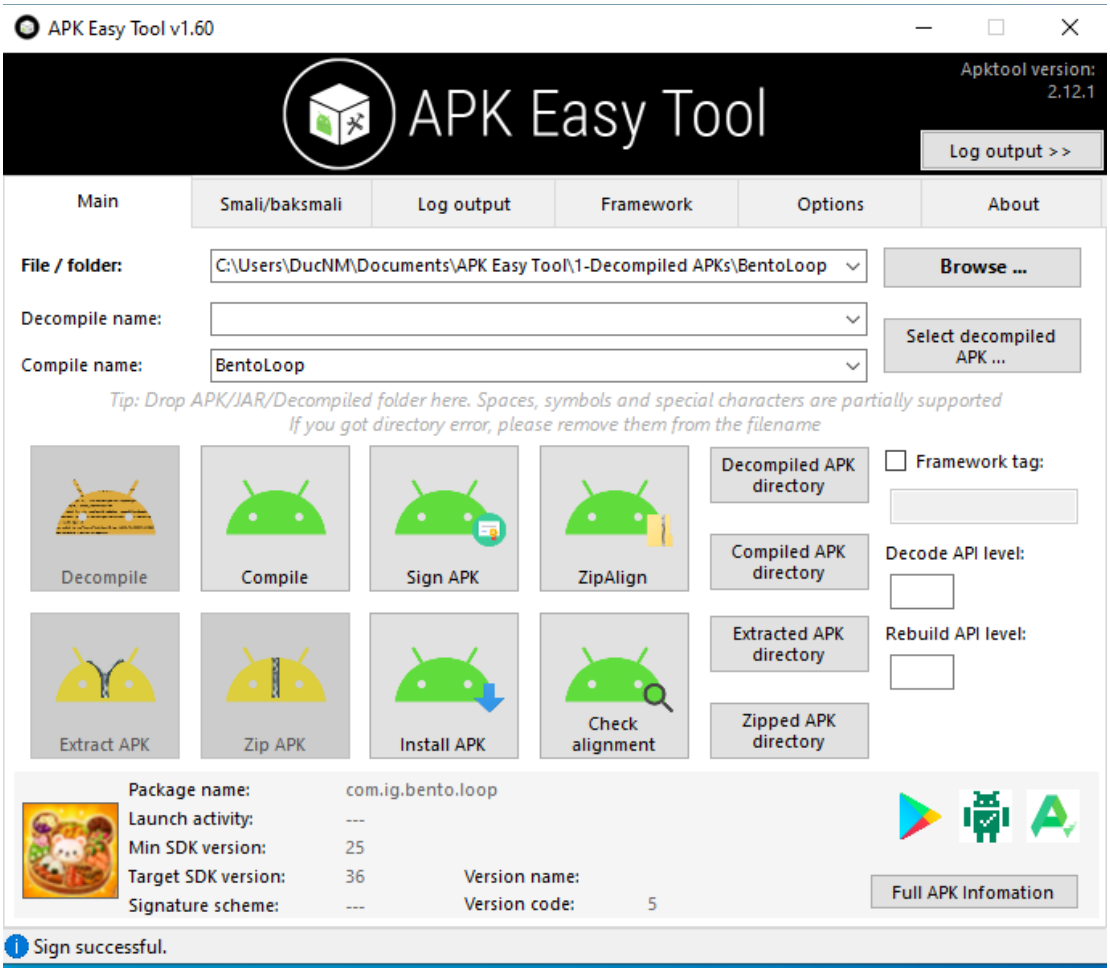
File 1.apk được đổi tên thành 1.apk.bak để giữ bản sao. Khi Compile lại, Apktool tạo file 1.apk mới từ framework đi kèm phiên bản 2.12.1. Ảnh thư mục cho thấy file mới có dung lượng 4.643 KB, còn bản sao cũ là 3.845 KB. Sau bước này, quá trình build tiến tới tạo và ký APK thành công.



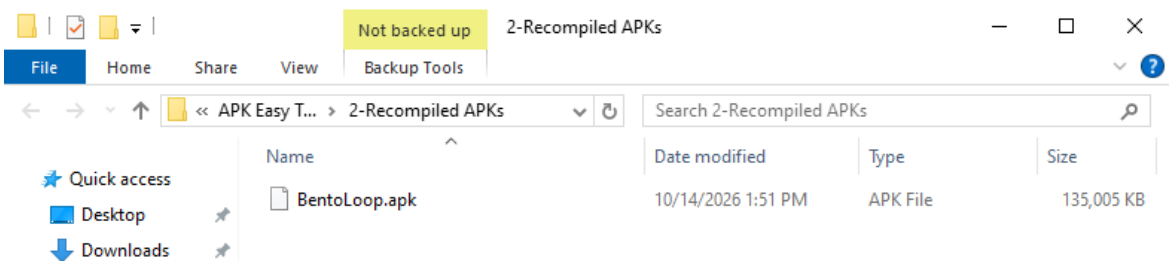
Hình 23. Framework mới được tạo bên cạnh bản sao 1.apk.bak

8.3 Lấy APK đã build và ký

APK Easy Tool hiển thị Sign successful sau quá trình build và ký. Bấm Compiled APK directory để mở thư mục 2-Recompiled APKs. File đầu ra là BentoLoop.apk, có dung lượng hiển thị 135.005 KB. Đây là APK dùng để kiểm thử bản đã sửa.



Hình 24. APK Easy Tool hoàn tất và báo Sign successful

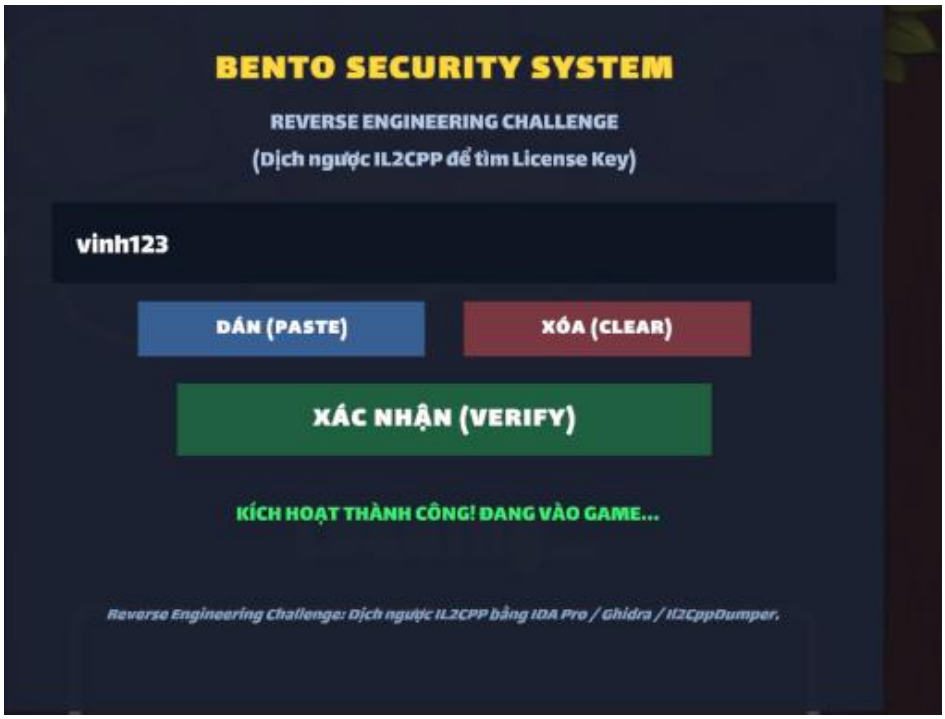


Hình 25. BentoLoop.apk trong thư mục 2 Recompiled APKs

APK được chuyển từ máy ảo sang máy thật để đưa lên thiết bị Android. Khi kiểm thử, cần dùng bản APK vừa build và bắt đầu từ trạng thái chưa kích hoạt, tránh nhầm với trạng thái được lưu từ lần nhập key đúng trước đó.

8.4 Kiểm thử với chuỗi khác key đã tìm được

Trên bản APK mới, nhập vinh123 rồi bấm XÁC NHẬN (VERIFY). Ứng dụng hiển thị KÍCH HOẠT THÀNH CÔNG! ĐANG VÀO GAME... dù chuỗi này khác BENTO-EASY-2025. Ảnh dưới được ghi nhận từ APK đã sửa, còn ảnh ở mục 6.3 thuộc APK gốc.



Hình 26. APK đã sửa báo kích hoạt thành công với chuỗi vinh123

Kết quả phù hợp với việc VerifyLicense trả về 1 ngay khi được gọi. Thử nghiệm xác nhận phần kiểm tra license tại hàm này đã được vòng tránh trên bản APK build lại. Phạm vi sửa là thư viện arm64-v8a và luồng kiểm tra license đã quan sát; thử nghiệm không đánh giá những chức năng khóa khác trong trò chơi.

Bản ứng dụng	Chuỗi thử nghiệm	Kết quả ghi nhận
APK gốc	BENTO-EASY-2025	Báo kích hoạt thành công và vào Level 1
APK đã sửa	vinh123	Báo kích hoạt thành công và đang vào game

9 Nhận xét và phương án phòng tránh

9.1 Nguyên nhân cơ chế có thể bị vòng tránh

Key BENTO-EASY-2025 được lưu dưới dạng chuỗi và so sánh trong thư viện phía client. Dữ liệu dump giúp xác định tên phương thức, từ đó lần đến phép so sánh và nội dung key. Nhánh kiểm tra mảng XOR không bảo vệ được nhánh chấp nhận chuỗi trực tiếp đang tồn tại song song.

Việc sửa hai lệnh đầu VerifyLicense cho thấy quyết định chấp nhận license phụ thuộc vào một giá trị trả về có thể thay đổi trong mã client. APK sau khi ký lại vẫn chạy được trong thử nghiệm và chấp nhận chuỗi khác key. Kết quả này cho thấy cơ chế bảo vệ đã quan sát chưa ngăn được cách can thiệp đã thực hiện.

9.2 Các biện pháp giảm rủi ro

Với ứng dụng có máy chủ, quyền sử dụng nên được xác minh theo tài khoản hoặc license trên máy chủ. Các thao tác cần quyền phải tiếp tục được kiểm tra tại nơi cung cấp dữ liệu hoặc dịch vụ. Cách này hạn chế việc sửa một màn hình hoặc một hàm client để nhận toàn bộ quyền sử dụng.

Không đưa key chung và nhánh chấp nhận thử nghiệm vào bản phát hành. Nếu cần hoạt động offline, license có thể được ký số: client giữ khóa công khai để kiểm tra, còn khóa riêng phát hành license được giữ ngoài ứng dụng. Cơ chế này giúp tránh nhúng một bí mật chung có thể đọc trực tiếp từ APK.

Kiểm tra chữ ký ứng dụng, tính toàn vẹn thư viện và phát hiện môi trường can thiệp có thể bổ sung khả năng phát hiện APK bị sửa. Làm rối mã giúp tăng chi phí phân tích nhưng không thay thế việc xác minh quyền ở phía tin cậy. Với môi trường offline, vẫn cần tính đến khả năng mã kiểm tra bị patch và lựa chọn mức bảo vệ phù hợp.

9.3 Kết luận thực hành

Bài thực hành hoàn thành việc tìm key BENTO-EASY-2025 và kiểm chứng trên APK gốc. VerifyLicense được sửa bằng hai lệnh ARM64 để trả về thành công, sau đó thư viện được xuất, thay vào thư mục đã Decompile và build lại bằng APK Easy Tool. Lỗi công cụ được xử lý bằng Apktool 2.12.1, bỏ tham số use aapt2 và làm mới framework. APK đã ký báo kích hoạt thành công với chuỗi vinh123.

Tài liệu tham khảo

Yêu cầu báo cáo và hướng dẫn APK Easy Tool được cung cấp trong bài thực hành.

Apktool 2.12.1
https://github.com/iBotPeaches/Apktool/releases/tag/v2.12.1

Tham số build của Apktool
https://apktool.org/docs/2.x/cli-parameters/

Quản lý framework trong Apktool
https://apktool.org/wiki/in-depth/frameworks/

Android Developers, phần tử property trong manifest
https://developer.android.com/guide/topics/manifest/property-element

OWASP MASVS RESILIENCE 2, bảo vệ ứng dụng trước can thiệp mã
https://mas.owasp.org/MASVS/controls/MASVS-RESILIENCE-2/

Android Developers, tổng quan Play Integrity API
https://developer.android.com/google/play/integrity/overview